

UNIVERSE - A CAMPUS COMPANION

The Timetable Engine

How the automatic routine generator works, end to end

OR-Tools CP-SAT - FastAPI - openpyxl - Flutter - Supabase

Project	CSE-3240 (Project I) - Leading University, Sylhet
Team	Sherlocked
Module owner	Fahmid Alam (Architecture, Timetable Engine, Admin)
Advisor	Jaminur Rahman
Differentiator	Automatic department timetable generation (advisor-required)
Document	Defense reference + knowledge transfer

1. What Problem Does It Solve?

A university department schedules hundreds of class sessions every semester. Each must land on a (day, period, room) such that no teacher is in two places at once, no student cohort has overlapping classes, lab courses get lab rooms, and every teacher's day-off is respected. Doing this by hand takes days and still produces clashes.

UniVerse's engine takes the department's existing 'Main Distribution' spreadsheet (who teaches what, to which batch) and produces a clash-free weekly routine automatically, in 30-120 seconds, returning both a published routine (visible to every student and teacher in the app) and a formatted Excel workbook identical to the one the department already uses.

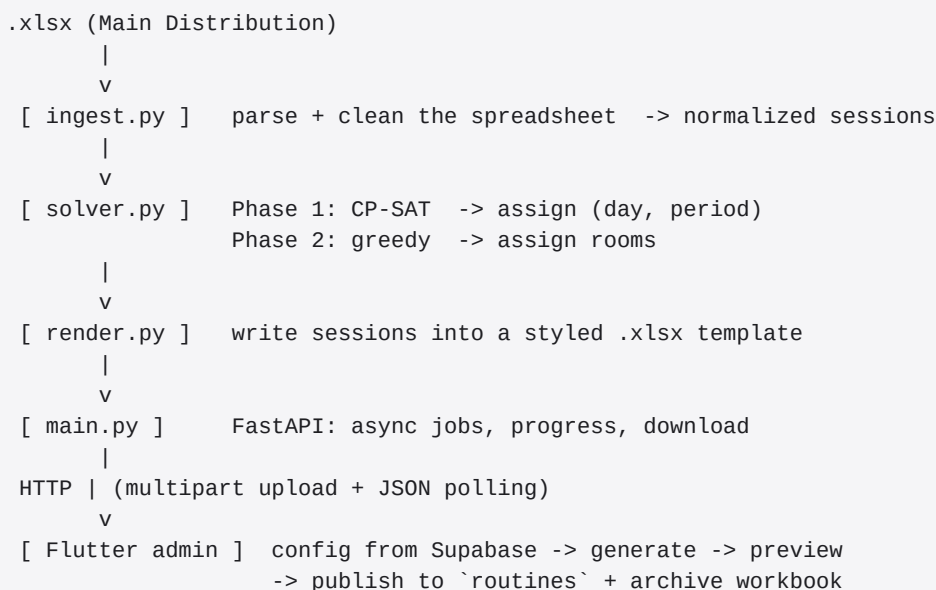
The advisor-required differentiator

Most campus apps are CRUD over a database. The constraint-solver timetable generator is what makes UniVerse a genuine engineering project: it turns a real combinatorial scheduling problem into a model an industrial solver (Google OR-Tools CP-SAT) can prove a solution for.

The pipeline at a glance

The engine is a small Python service of four cooperating modules plus a Flutter client that drives it. Data flows in one direction:

Figure 1 - module data flow



The four Python files + the Flutter layer

ingest.py	Reads the 'Course Distribution' sheet and converts each course offering into normalized Session records. Applies every real-world data-quality rule.
solver.py	The heart. Builds the CP-SAT constraint model (Phase 1) and the greedy room assignment (Phase 2). Also validates the result.
render.py	Clones a styled Excel template and writes the solved sessions into it, reproducing the department's exact routine format.
main.py	FastAPI wrapper. Exposes the pipeline over HTTP with an async job model so the app can show a live progress bar.

config.json	Default rooms, period grid, teacher off-days and solver weights. In production this is overridden by config built from Supabase.
Flutter admin	Service + controller + screens that pick the file, build config from the DB, drive the job, preview the result, and publish it.

2. ingest.py - Turning a Spreadsheet into Clean Data

Real spreadsheets are messy: columns drift, batches are stored as floats, project rows have no teacher, and some rows are service courses taught to other departments. ingest.py exists so the solver never sees any of that mess - it receives only clean, typed records.

Key data structure: the Session dataclass

Each course offering becomes TWO Session objects (occurrence 1 and 2).

```
@dataclass
class Session:
    sid: int          # unique id, used as the CP-SAT variable key
    code: str         # "CSE-1101"
    title: str
    teacher: str      # acronym, e.g. "EBH"
    batch: str        # "66"
    section: str      # "A", "B+C", "GED A", "WD"
    cohort: str       # "66-A" (batch + section)
    is_lab: bool      # needs a lab room?
    is_service: bool  # taught to non-CSE -> resource-only
    occurrence: int   # 1 or 2 (every course meets twice a week)
```

Why two sessions per offering? Department policy is Class/Week = 2 - every course meets twice weekly. Modelling each meeting as its own Session lets the solver place the two meetings independently (and then a soft constraint pushes them onto different days).

How columns are found (robust parsing)

The file's header lives on row 5, data starts on row 6. Columns are bound by matching header TEXT, never by fixed letter - because the real files occasionally shift a column. `_header_map()` reads row 5 into a {lowercased-header -> column-index} map, and `_col()` looks up a column by header prefix:

Tolerant lookup: accepts spelling/spacing variants seen in real files.

```
def _col(hmap, *prefixes):          # tolerant header lookup
    for p in prefixes:
        for k, v in hmap.items():
            if k.startswith(p.lower()):
                return v
    return None

cwk = _col(h, "class/week", "class /week", "class per week")
```

The data-quality rules (reverse-engineered from real files)

- Batch stored as float '66.0' -> normalized to '66' via `_norm_batch()`.

- Lab det
- CSE-11

Why 'service' sessions matter

A CSE teacher might also teach a GED course to Business students. The engine cannot ignore that class - the teacher is genuinely busy then. So it is solved as a resource-only booking: it holds teacher + room time during Phase 1, but is filtered out of the final CSE routine. This is the single most important correctness idea in ingest.

Output

`ingest_workbook()` returns a dict: cohorts (canonical render order, batch descending), sessions (list of dicts), teachers_used, excluded, warnings, and meta (counts). On the real Summer-2025 file this is ~343 offerings -> ~686 sessions across 68 cohorts, 76 teachers.

3. solver.py Phase 1 - The CP-SAT Constraint Model

What is CP-SAT and why use it?

CP-SAT (Constraint Programming - Satisfiability) is Google OR-Tools' constraint solver. You describe a problem as decision VARIABLES, HARD constraints that any valid solution MUST satisfy, and a SOFT objective to minimize. The solver then searches - using SAT solving, constraint propagation and parallel workers - for an assignment that satisfies all hard constraints while making the objective as small as possible. We never write the search algorithm ourselves; we only state the rules.

The mental model

Think of it as 'declarative scheduling'. We don't tell the computer HOW to build the timetable. We tell it WHAT a valid timetable is (the constraints) and WHAT a good one looks like (the objective), and OR-Tools figures out the rest - with a proof of feasibility.

The decision variables

For every session s and every valid (day d , period p) it could occupy, we create a boolean variable $x[s, d, p]$. It equals 1 if session s is scheduled on day d , period p - and 0 otherwise. This is the classic 'one boolean per possible placement' encoding.

One boolean per (session, day, period) candidate.

```
x[(s["sid"], d, p)] = model.NewBoolVar(f"x_{s['sid']}_{d}_{p}")
```

Variables are only created for VALID slots. `valid_slots(s)` pre-filters out the teacher's off-days, and Friday Period-4 (a department rule), and the online column. Pruning the variable space up front makes the model smaller and the solve faster.

Candidate slots are filtered before variables are even created.

```
def valid_slots(s):
    out = []
    tea_off = off_days.get(s["teacher"], set())
    for di, dname in enumerate(days):
        if dname in tea_off:          # teacher day-off
            continue
        for p in pidx:
            if friday_no_p4 and dname == "Friday" and p == 4:
                continue              # no Friday P4
            out.append((di, p))
    return out
```

Hard constraints (a valid timetable MUST satisfy all)

H1 - Exactly once	Every session is placed in exactly one (day, period): <code>model.AddExactlyOne(all x for that session)</code> . If a session has zero candidate slots, the engine raises a clear error instead of failing silently.
H2 - Teacher	For each (day, period), a teacher has at most one class: for every teacher group, <code>sum(x at that slot) <= 1</code> . This prevents a teacher being double-booked.
H3 - Cohort	For each (day, period), a student cohort has at most one class: <code>sum(x for that cohort) <= 1</code> . Prevents student clashes.
H4/H5 - Room capacity	Per (day, period) the number of lab sessions <code><=</code> number of lab rooms, and theory sessions <code><=</code> theory rooms. This guarantees Phase 2 (room assignment) can ALWAYS succeed - it never over-commits a slot.

H2/H3 are built by grouping sessions with `by_teacher` and `by_cohort` dicts, then for each (day, period) summing the relevant variables and bounding the sum by 1.

Clash + capacity constraints, built slot by slot.

```
for di in range(len(days)):
    for p in pidx:
        for group in by_teacher.values():
            terms = [x[(s["sid"], di, p)] for s in group if ...]
            if len(terms) > 1:
                model.Add(sum(terms) <= 1)      # H2
        for group in by_cohort.values():
            ...                                  # H3 (same shape)
# H4/H5: lab vs theory counts bounded by available rooms
model.Add(sum(lab_terms) <= n_lab)
model.Add(sum(th_terms) <= n_theory)
```

Soft constraints (preferences - minimized, not forced)

Hard constraints decide if a timetable is legal. Soft constraints decide if it is GOOD. Each soft rule contributes a weighted penalty term; the solver minimizes the sum of all penalties. Weights come from config (defaults: different_days 8, compactness 3, late_slot 1), so the relative importance is tunable without code changes.

S1 - Different days (w=8)	The two weekly meetings of one course should fall on different days. For each day a boolean `same` is forced to 1 if both meetings land that day, and `same` is penalized. Highest weight - spreading a course across the week matters most.
S3 - Compactness (w=3)	A cohort's classes should cluster into fewer distinct days (so students don't come in for a single class). A `used[cohort,day]` boolean is set if the cohort has any class that day, and each used day is penalized.
S7 - Late slot (w=1)	Avoid the last in-person period when possible; each session placed in the final period adds a small penalty.

S1 in detail - the linearization trick

'Both meetings on the same day' is naturally a logical AND, which a linear solver can't express directly. It is linearized: with y_1 = sum of meeting-1's variables on day d, and y_2 = sum of meeting-2's, the constraint $y_1 + y_2 - 1 \leq \text{same}$ forces `same` to 1 whenever both are present ($1 + 1 - 1 = 1 \leq \text{same}$). The minimizer keeps `same` at 0 otherwise.

Soft rules become weighted penalty terms in one objective.

```
same = model.NewBoolVar(...)
model.Add(sum(y1) + sum(y2) - 1 <= same)
penalties.append(w["different_days"] * same)
...
model.Minimize(sum(penalties))
```

Running the solver

8 workers search in parallel under a time budget.

```
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = time_limit_s    # default 60s
solver.parameters.num_search_workers = 8                # parallel search
status = solver.Solve(model)
```

- max_time_in_seconds: a hard budget. If the optimum isn't proven in time, the best FEASIBLE solution found so far is returned - good enough for scheduling.

- num_se
is why a

Reading the solution back

After solving, for each session we scan its candidate slots and pick the one whose variable the solver set to 1 - that (day, period) is the session's placement. These go into a `placed` list handed to Phase 2.

Extract chosen placements from the solved model.

```
for s in sessions:
    for (d, p) in slots_for[s["sid"]]:
        if solver.Value(x[(s["sid"], d, p)]) == 1:
            placed.append({"s": s, "day": d, "period": p})
            break
```

4. solver.py Phase 2 - Greedy Room Assignment

Phase 1 decided WHEN each session runs but not WHERE. Because H4/H5 already guaranteed no slot has more lab/theory sessions than rooms, a simple greedy pass is provably sufficient - no backtracking needed. This is a deliberate design choice: keep the hard combinatorial part (time) in the solver, and solve the easy part (rooms) cheaply.

The algorithm

- Sort placements so labs are assigned first (they have the scarcer room pool).

- Keep a

First-fit room selection per slot.

```
placed.sort(key=lambda q: (not q["s"]["is_lab"],)) # labs first
def pick(pool, d, p):
    for r in pool:
        if r not in busy.get((d, p), set()):
            return r
    return None
room = pick(pool, d, p) or "TBA"
busy.setdefault((d, p), set()).add(room)
```

Building the output rows

Each placed session becomes a row matching the Supabase `routines` columns: day, period, time_start/end, subject, subject_code, teacher_name/code, room, batch, section, semester, is_lab, is_service. Service rows are kept separate (not published). A helper semester_for() maps the batch number to a 1-8 display semester (newest batch = semester 1).

Batch number -> displayed semester, matching the app's convention.

```
def semester_for(batch):
    # newest (highest) batch -> semester 1; each older batch +1
    return min(8, max(1, max_batch - int(batch) + 1))
```

Phase 3 - Validation (defense-in-depth)

Even though the constraints should guarantee correctness, _validate() independently re-checks the FINAL rows and counts any violation. This is the 'trust but verify' layer - if a refactor ever broke a constraint, the report would catch it. Every count should be 0.

teacher_clashes	/	Two rows sharing a (teacher cohort room, day, period). Must be 0.
cohort_clashes	/	
room_clashes		
dayoff_violations		A class scheduled on a teacher's day-off.
friday_p4_violations		A class in Friday's forbidden Period 4.
lab_room_violations		A lab course placed in a non-lab room.
unplaced_rooms		Sessions left as 'TBA'.
offerings_not_twice		Any offering not placed exactly twice (Class/Week = 2 check).
ok		True only if every count above is 0 - the headline pass/fail shown in the app.

5. render.py - Reproducing the Department's Excel Routine

The department already publishes routines in a specific styled Excel format (banner, merged headers, a vertical BREAK column, bus-time rows). Rather than build that from scratch, render.py clones a saved TEMPLATE workbook and writes sessions into the correct cells, preserving every bit of formatting.

How cells are located

- Each day is a separate worksheet (Sunday..Saturday).

- Cohorts
one can

Period idx -> column, then write the session string.

```
SESSION_COLS = [4, 5, 6, 8, 9, 10, 11] # D,E,F,H,I,J,K (G=break skipped)
pcol = {int(p["idx"]): column_index_from_string(p["col"])}
    for p in config["periods"]}
text = f"{s['subject_code']} {s['teacher_code']} {s['room']}"
ws.cell(row=rr, column=col, value=text)
```

A `_safe_clear()` helper avoids writing to non-anchor cells of merged ranges (openpyxl raises on those). The finished workbook is returned as raw bytes, ready to stream over HTTP or upload to Supabase Storage.

openpyxl - the Excel library

openpyxl reads and writes .xlsx files in pure Python. ingest.py uses it with `data_only=True` (so it reads computed values, not formulas); render.py uses it to load the template, edit cells while keeping styles, and save back to a bytes buffer.

6. main.py - The FastAPI Service & Async Job Model

A CP-SAT solve takes 30-120 seconds - too long for a single blocking HTTP request (the client would time out and the user would stare at a frozen screen). So main.py uses an asynchronous job model: start the work in a background thread, hand back a job_id immediately, and let the client poll for progress.

The endpoints

POST <code>/api/timetable/generate</code>	Accepts the .xlsx (multipart) + optional config JSON + time_limit. Starts a background Thread running the pipeline and returns { job_id } at once.
GET <code>/status/{job_id}</code>	Returns { state, progress }. state cycles queued -> ingesting -> solving -> rendering -> done (or failed). progress is 0.0..1.0 for the progress bar.
GET <code>/result/{job_id}</code>	After done: the full payload - rows, stats, validation, and the ingest report (cohorts, excluded, warnings).
GET <code>/download/{job_id}</code>	Streams the rendered .xlsx with a proper filename and content-type.
GET <code>/</code>	Health check - used to confirm the server is reachable.

The background worker

`_run_job()` runs the whole pipeline and updates the shared JOBS dict as it goes. The solver reports its own 0.5..0.96 internal progress, which is mapped into a 0.2..0.95 band so the client's bar moves smoothly across ingest, solve and render.

Start work in a thread; the client polls JOBS via /status.

```
JOBS: dict[str, dict] = {}                                # job_id -> state/progress/result

def _run_job(job_id, file_bytes, config_override, time_limit_s):
    job["state"] = "ingesting"; dataset = ingest.ingest_bytes(file_bytes)
    cfg = solver.load_config(override=config_override)
    job["state"] = "solving"
    result = solver.solve(dataset, cfg, time_limit_s, progress=...)
    job["state"] = "rendering"
    job["file"] = render.render_bytes(result["rows"], dataset["cohorts"], cfg)
    job["state"] = "done"

Thread(target=_run_job, args=(...), daemon=True).start()
```

Design note: in-memory jobs

JOBS is a plain dict in memory - no database. This is intentional for a single-instance demo deployment (Railway/Render free tier). A production multi-instance deployment would move job state to Redis or a DB so any worker can answer a poll. Stating this trade-off openly is a strong defense point.

Config normalization - one solver, two input shapes

`solver.load_config()` accepts EITHER the engine's own config.json shape OR the DB-sourced shape (rooms[], faculty[], settings{}) sent by Flutter, and `_normalize_config()` folds both into one internal form. This is why the same engine runs from the CLI (file config) and from the app (database config) with no code changes.

7. config.json - The Knobs

Everything institution-specific lives in config, not code - so the engine generalizes to any department by editing data only.

days	The week (Sunday..Saturday).
periods	The time grid: each period has idx, label, start, end, and Excel column. Idx 1-6 are in-person; idx 7 (19:00) is the online column, excluded from solving.
friday_no_p4	Department rule: Friday has no Period 4 (prayer break).
rooms	Each room: name, building, is_lab, is_gallery. Split into lab vs theory pools.
teachers	Acronym -> full_name + off_days. Off-days become hard constraints.
weights	Soft-constraint weights: different_days, compactness, late_slot - tune solution character without touching code.
semester_label	e.g. 'Summer 2025' - used for filenames and the run record.

8. How It Binds to the Flutter App

The engine is a separate Python service; the app talks to it over HTTP and follows the project's strict layering: Screen -> Controller -> Service -> (HTTP / Supabase). Screens never touch HTTP or Supabase directly.

The layers

Admin config screens	Manage Rooms / Manage Faculty / Settings - admins curate rooms, teacher off-days, periods and weights, stored in Supabase tables <code>timetable_rooms</code> / <code>timetable_faculty</code> / <code>timetable_settings</code> .
timetable_config_service.dart	The only Supabase layer for those three config tables. <code>buildEngineConfig()</code> assembles the JSON the engine consumes (active rooms, faculty off-days, settings).
timetable_engine_service.dart	The only layer that talks to the FastAPI engine. <code>generate()</code> , <code>pollStatus()</code> , <code>fetchResult()</code> , <code>downloadWorkbook()</code> - plus <code>publishToRoutines()</code> and <code>recordRun()</code> on Supabase.
timetable_gen_controller.dart	ChangeNotifier state for the Generate screen: pick file, build config, start job, poll every 2s (max ~4 min), hold result, download/open, publish.
Generate Timetable screen	ListenableBuilder UI: pick .xlsx, watch the progress bar, preview stats + validation + warnings, then publish.

The end-to-end sequence

From file pick to published routine.

```

1. Admin picks Main_Distribution.xlsx          (file_picker, withData)
2. controller.generate():
   config = configService.buildEngineConfig() (rooms+faculty+settings from DB)
   jobId = engineService.generate(bytes, config) POST /generate
3. poll loop every 2s: engineService.pollStatus(jobId) GET /status
   -> update progress bar (0.0 .. 1.0)
4. on done: engineService.fetchResult(jobId)   GET /result
   -> RoutineEntry rows + stats + validation + report (preview)
5. controller.publish():
   publishToRoutines(rows) -> clears old rows per batch, inserts new
   uploadWorkbook(bytes)  -> archives .xlsx in `timetables` bucket
   recordRun(...)         -> logs run in `timetable_runs`

```

Idempotent publish

`publishToRoutines()` first DELETES existing `routines` rows for each generated batch, then inserts the new ones. Re-running a generation never creates duplicates - the published routine is always a clean replacement.

Once published, the rows live in the same `routines` table the student and teacher routine screens already read - so the generated timetable instantly appears for every user, with no extra wiring. The Android emulator reaches a locally-run engine at `http://10.0.2.2:8000`; in production the engine is deployed on Railway/Render (Procfile: `uvicorn main:app`).

9. Packages, Algorithms & Defense Cheat-Sheet

Python packages

ortools (CP-SAT)	Google's constraint solver. Provides CpModel (variables/constraints), CpSolver (parallel search). The engineering core.
fastapi	Async web framework - defines the HTTP endpoints, parses multipart uploads, serializes JSON.
uvicorn	The ASGI server that actually runs the FastAPI app (production start command).
openpyxl	Reads/writes .xlsx - parsing the distribution file and rendering the routine.
python-multipart	Lets FastAPI accept the file upload (multipart/form-data).

Flutter / Dart packages

http	Multipart upload + JSON polling against the engine.
supabase_flutter	Config tables, publishing to `routines`, storage, run records.
file_picker	Picks the .xlsx with its bytes (withData: true).
open_filex / path_provider	Saves and opens the downloaded workbook on-device.

Algorithms & techniques - name them in the defense

- Constraint Programming / SAT solving (CP-SAT) for time assignment.

- Boolean

Likely defense questions - quick answers

Why CP-SAT, not greedy/genetic?	Scheduling has hard feasibility constraints; CP-SAT can PROVE a clash-free solution exists (or report infeasible) and optimizes preferences - greedy/GA give no guarantee.
What if no solution exists?	The solver returns INFEASIBLE; we raise a clear message ('a teacher has more sessions than free slots'). H1 also fails fast if a session has zero candidate slots.
Why two phases?	Time placement is the hard combinatorial part; rooms are easy once per-slot capacity is bounded. Separating them keeps the model small and fast while staying provably correct.
How are teacher day-offs respected?	They prune candidate slots BEFORE variables exist, so an off-day class is literally impossible, not just penalized.
How fast / how big?	~343 offerings, ~686 sessions, 76 teachers, 68 cohorts, 27 rooms; solves in 30-120s with 8 parallel workers under a time budget.

Does it scale to other departments?	Yes - all institution data is config (rooms, periods, off-days, weights). No code change needed.
--	--

Summary: ingest cleans the spreadsheet -> CP-SAT proves a clash-free (day, period) assignment honoring teacher/cohort/room/off-day rules while minimizing weighted preferences -> a greedy pass fills rooms -> openpyxl renders the department's exact Excel format -> FastAPI serves it as an async job -> the Flutter admin layer configures, drives, previews and publishes it into the same `routines` table every student and teacher already reads.